National College of Ireland

Project Submission Sheet

Student Name:	Meenakshi		
Student ID:	24235415		
Programme:	Msc in Cloud Computing	**Year:**	2025-2026
Module:	Scalable Cloud Programming		
Lecturer:	Mr. Vikas Sahni		
Submission Due Date:	1st April 2026		
Project Title:	Smart Parking Management System		
Word Count:	……		

Academic Integrity Certificate Verification Code: Q6lveHvE6b

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: 🔗 ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature:	Meenakshi ……
Date:	1st April 2026 ……

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**

5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date
Meenakshi	Msc in Cloud Computing	1st April 2026

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

Academic Integrity Acknowledgement Supplement

The red square below indicates certificate number, please put this in the above field



NCI
Coláiste Náisiúnta hÉireann | National College of Ireland

This certificate is awarded to:

Meenakshi .

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

January 18, 2026

Certificate Verification Code: Q6lveHvE6b

Course Code: MSCCLOUD

epigeum

Grok

Academic Integrity Subject Guide or how to access the site:
https://libguides.ncirl.ie/academicintegrity/epigeumforstudents
Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI

Smart Praking Management System

Meenakshi
StudentId: 24235415
Scalable Cloud, MSc in Cloud Computing
National College of Ireland Dublin, IRELAND
Email: x24235415@student.ncirl.ie. URL: www.ncirl.ie
Deployed Application URL: smart-parking.us-east-1.elasticbeanstalk.com

***Abstract*—The main perpective of this project is to develop a cloud-based application that implements multiple web services and utilize them together. Actually basic idea behind this project is to develop a front-end application that accepts user input and interact with different API to achieve different purpose. In this project, I created my own API for password strength checking, another API provided by a classmate for converting HTML data into PDF format, and also used another public API to fulfill the requirements of the project.**

The application provides an opportunity for users to manage vehicle information such as adding and deleting data. The most interesting part of the project is that when a vehicle is deleted from the application, it will generate a PDF document containing the vehicle information. The application utilizes another public API for HTML to PDF conversion.

The application is developed by utilizing RESTful APIs and deployed on a cloud platform so that it is accessible online and supports multiple users. The project helped me understand how a real-world cloud application is developed by utilizing multiple APIs instead of a single application.

I. Introduction

In the current world, the application is not developed as one single system. Instead, an application is made to up of multiple small services that work together. This makes the application flexible, easier to maintain, and scalable is when the number of users increases. This also brings some to challenges, such as communication between the services and the various APIs.

The motivation is behind the project was to learn the process of building such systems in real life. Instead of including all the functionalities in one backend, the system is divided into different parts. For example, the password checking is performed by one API, while PDF generation is performed by another API. All the functionalities are an performed separately. This helps in understanding the process of building cloud-based applications.

II. Project Objectives

The major to aim of this project is to create a functional cloud-based application that in corporates different web services. The first aim of the project is to create a front-end application that allows users to input vehicle is information. The second aim of the project is to create my own API that and tests the strength of a password entered by the user.

Another significant aim of the project is to incorporate a classmate's API, which converts HTML data into a PDF file. This API is used in the project when a vehicle is deleted from the system. The API generates a downloadable PDF file that contains vehicle information. Apart from this one, a public API is also incorporated to fulfill the requirements of the project.

The last aim of the to project is to host the application on a cloud environment and learn some of the basic concepts of scalability, API communication, and cloud hosting.

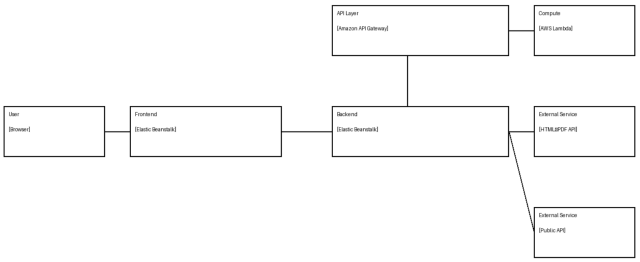


Fig. 1. Block Diagram

III. Project Specification and Requirements

Functional Requirements

This system provides the user with the ability to perform simple operation on the data related to the vehicles. A user can add a vehicle is to the system by providing the required details such as the name, number, and any of other relevant information. During the process of adding the data, the api of password entered by the user is validated using the custom API.

Also, the user can delete the vehicle. During the deletion process, something is extra is done by the system. It sends the data related to the vehicle to the HTML to PDF API. It generates a PDF document with the details related to the deleted vehicle. The user can download this document.

This system also makes sure that all the APIs are working well together.

Non-Functional Requirements

Apart from this functional requirements, the system also considers some of the important non-functional requirements. One of the most important requirements is to scalability. Since the application is hosted on the cloud, it should be able an to handle greater number of users if required.

Another important requirement is that the application should be accessible anytimewith via the internet. The performance of the application is also important so that the responses are fast and do not cause any delay the user experience.

The application should also simple and easy to use so that the user does not face any difficulty while using it.

IV. System Overview

The system is designed a manner that different components perform different functions rather than two large system performing all functions. The front-end application is used to perform functions related to an interacting with the user. It receives input and sends requests to the backend.

The backend API receives the data and performs operations like password validation. It uses other APIs for extra features like PDF creation. All these components communicate with each other by sending HTTP requests.

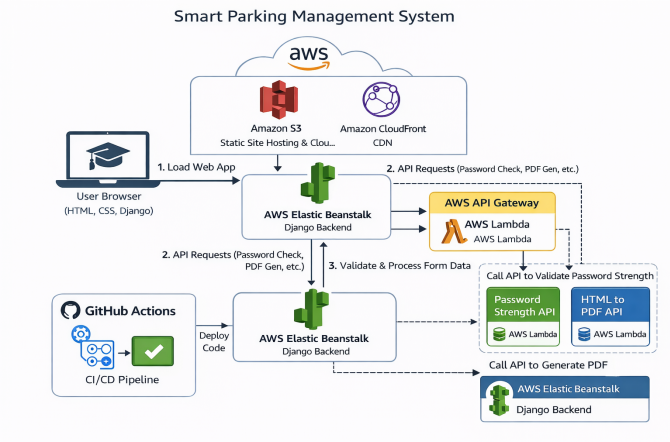


Fig. 2. Architecture Diagram

V. Design Approach

The application follows a RESTful design pattern. This means that all communication between the frontend and backend occurs via APIs. The APIs have specific roles to play, and they are loosely coupled. This makes it easier to modify or update one API without affecting the others.

The backend API is simple and stateless. It receives an API request, performs the operation, and then sends a response. It does not retain any information in the session. This design pattern will help in creating a scalable application.

VI. Scalability Design

Scalability is another significant aspect of cloud-based applications. In this project, the system is designed in a manner such that it can support more users if needed in the future. Since the system is stateless, multiple instances of the API can be run at any given time.

Additionally, since the system is divided into separate services, they can be scaled individually. If, for instance, the PDF service is more utilized, it can be scaled individually without interfering with other services.

The use of a cloud-based system will enable the implementation of features such as load balancing and auto-scaling in future improvements.

VII. Frontend Application

The role of the frontend application is to interact with the user. It is a simple interface in which users can input information regarding the vehicle and can take actions like add and delete.

When users input information, the frontend sends information to the backend API. It also displays information received from the API, like password strength and confirmation messages.

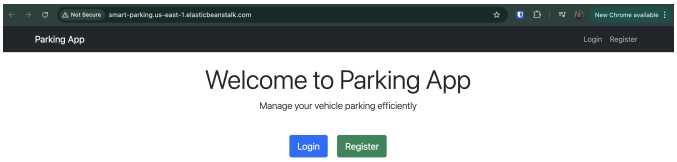


Fig. 3. Landing Page

VIII. Backend API (Password Strength Checker)

The most important API developed in this project is the password strength checker. This API receives a password from the frontend. Then it checks whether the password is strong or weak.

It is checked by following some simple rules such as length, numbers, and special characters. Finally, it returns the result in the form of weak, medium, or strong. This is for security purposes.

IX. Integration with Other APIs

In this project, several APIs are integrated to fulfill the requirement. The classmate's API is used to convert HTML content into a PDF file. When the vehicle is deleted, the data is converted into HTML format and sent to the API to obtain the PDF file.

Another public API is used to obtain the data and fulfill the requirement of using the external services. These APIs are integrated using HTTP requests, and the proper responses are handled.

X. Working Flow

The process of the system begins with the entry of vehicle details by a user. The password entered by the user is validated using the password API, and if everything is correct, the data is saved.

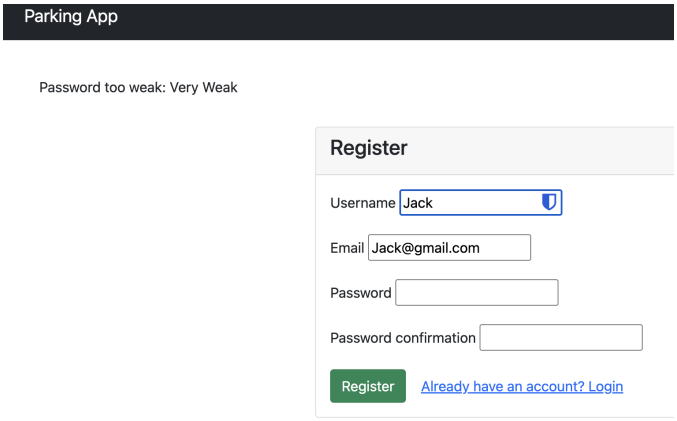


Fig. 4. Implementation of Password Strength API



Fig. 5. Implementation of HTML to PDF API

In case a user deletes a vehicle, the system takes this data and converts it to HTML format, which is further sent to the PDF conversion API, and a PDF file is created, which is then sent to the user.

XI. Deployment

The application will be deployed on a cloud platform so that it can be accessed anywhere. The backend API will be hosted online, and the frontend will be connected using API calls.

The application will be managed using a cloud platform, and scalability features will also be provided.

XII. CI/CD

http://smart-parking.us-east-1.elasticbeanstalk.com/ The basic CI/CD process is implemented in this project. Whenever changes are made in the code and pushed to GitHub, the build and deployment of the project are done. This helps in automating the process, which otherwise would have required manual intervention.

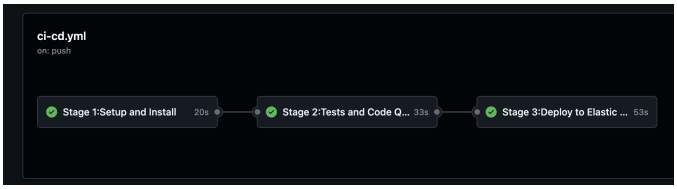


Fig. 6. CICD

XIII. Conclusion

This project helped me gain knowledge about how an application is actually developed in the real world by using multiple services. Initially, I thought it was an easy project, but after I started working on the integration of APIs, I realized that there are many small things to take care of.

One of the things I learned from this project is that even if all the APIs are working individually, it does not mean that integrating them is an easy task. Sometimes problems may occur due to the data format, and sometimes the output may not be as expected. Debugging is required in such cases.

Moreover, working on cloud deployment helped me gain knowledge about how a scalable application is developed. Though this project is small in nature, the same concept can be used for a big application also.

I gained knowledge about APIs, back end development, and cloud deployment from this project. I also gained knowledge about how multiple services can be integrated into a single application, which is very important in cloud computing.

[1] [2] [3] [4] [5] [6] [7] [8]

References

[1] Amazon Web Services, "Aws well-architected framework: Security pillar," AWS Documentation, 2024, [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/. [Accessed: Nov. 15, 2025].

[2] R. P. L. Buse and T. Zimmermann, "Information needs for software development analytics," in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, Zurich, Switzerland, 2012, pp. 987–996.

[3] M. Felderer, M. Fournire, D. Hutter, C. Pecheur, B. Preschern, M. Ramler, and A. Seidl, "Security testing: A survey," *Advances in Computers*, vol. 101, pp. 1–51, 2016.

[4] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, Irvine, CA, 2000.

[5] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[6] K. Hashizume, D. G. Rosado, E. Fernández-Medina, and E. B. Fernandez, "An analysis of security issues for cloud computing," *Journal of Internet Services and Applications*, vol. 4, no. 1, pp. 1–13, 2013.

[7] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, Hyderabad, India, 2014, pp. 435–445.

[8] C. Schafer, "Python django tutorial: Deploying your application to a linux server," YouTube, Sep. 2018, [Online]. Available: https://www.youtube.com/watch?v=Sa$_k$$QheCnds$.[$Accessed$: $Nov.10, 2025$].